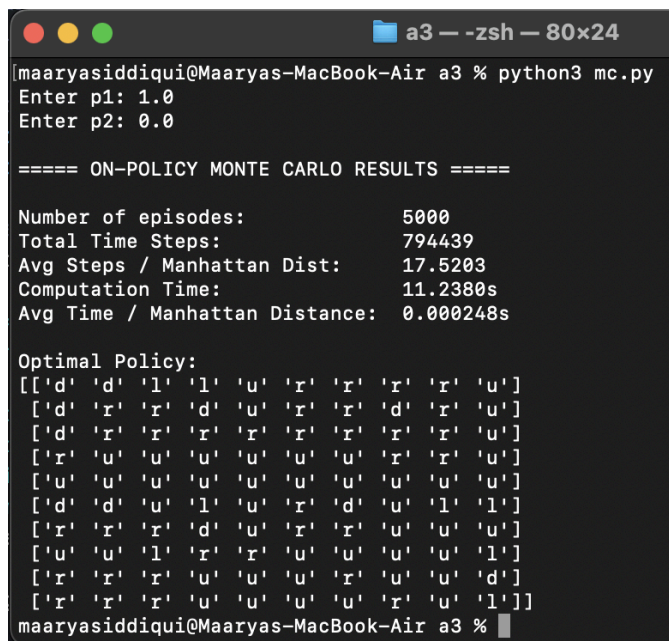


Q3 - Monte Carlo, SARSA, Q-Learning

The Monte Carlo, SARSA and Q-Learning algorithms were implemented using Python. The following commands were used to view the statistics for each algorithm individually. Note that for the three algorithms mentioned below, $\alpha=0.1$ and $\epsilon=0.1$:

```
python3 mc.py
python3 sarsa.py
python3 q.py
```



```
a3 --zsh-- 80x24
maaryasiddiqui@Maaryas-MacBook-Air a3 % python3 mc.py
Enter p1: 1.0
Enter p2: 0.0

===== ON-POLICY MONTE CARLO RESULTS =====

Number of episodes:          5000
Total Time Steps:            794439
Avg Steps / Manhattan Dist:  17.5203
Computation Time:            11.2380s
Avg Time / Manhattan Distance: 0.000248s

Optimal Policy:
[['d' 'd' 'l' 'l' 'u' 'r' 'r' 'r' 'r' 'u']
 ['d' 'r' 'r' 'd' 'u' 'r' 'r' 'd' 'r' 'u']
 ['d' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'u']
 ['r' 'u' 'u' 'u' 'u' 'u' 'u' 'r' 'r' 'u']
 ['u' 'u' 'u' 'u' 'u' 'u' 'u' 'u' 'u' 'u']
 ['d' 'd' 'u' 'l' 'u' 'r' 'd' 'u' 'l' 'l']
 ['r' 'r' 'r' 'd' 'u' 'r' 'r' 'u' 'u' 'u']
 ['u' 'u' 'l' 'r' 'r' 'u' 'u' 'u' 'u' 'l']
 ['r' 'r' 'r' 'u' 'u' 'u' 'r' 'u' 'u' 'd']
 ['r' 'r' 'r' 'u' 'u' 'u' 'u' 'r' 'u' 'l']]
maaryasiddiqui@Maaryas-MacBook-Air a3 %
```

Monte Carlo Analysis

Over 5000 episodes, the total time steps the agent took was 794,439 before converging. This is a significant amount of time the agent took to learn where the goal is located (top-right cell). Monte Carlo achieved a normalized step count of 17.5, meaning the agent required about 17.5 moves for every unit of Manhattan distance to the goal. This could be due to the epsilon greedy exploration, stochastic transitions and the fundamental limitation of this algorithm where it only updates Q-values at the end of the episode, causing the agent to move inefficiently around the grid.

In addition, the computation time for all 5000 episodes was 11.3 seconds, highlighting how computationally expensive this algorithm is. Furthermore, the normalized time of 0.000248s reflects the per-step computational expense of this algorithm. While this absolute value is small, it is the largest when compared to SARSA and Q-Learning (see below).

```

a3 - zsh - 80x24
maaryasiddiqui@Maaryas-MacBook-Air a3 % python3 sarsa.py
Enter p1: 1.0
Enter p2: 0.0

===== SARSA RESULTS =====

Number of episodes:          5000
Total Time Steps:            64723
Avg Steps / Manhattan Dist:  1.4455
Computation Time:            0.9742s
Avg Time / Manhattan Dist:   0.000022s

Optimal Policy:
[['d' 'd' 'd' 'd' 'u' 'r' 'r' 'r' 'u']
 ['r' 'd' 'r' 'd' 'u' 'r' 'd' 'd' 'u']
 ['r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'u']
 ['r' 'u' 'u' 'u' 'u' 'r' 'u' 'l' 'u']
 ['u' 'u' 'u' 'u' 'u' 'u' 'u' 'u' 'u']
 ['r' 'r' 'u' 'l' 'u' 'r' 'r' 'u' 'l' 'l']
 ['u' 'u' 'u' 'l' 'u' 'r' 'r' 'u' 'l' 'u']
 ['r' 'r' 'r' 'r' 'r' 'u' 'u' 'l' 'l']
 ['u' 'u' 'r' 'u' 'u' 'u' 'u' 'u' 'l']
 ['r' 'r' 'r' 'u' 'u' 'u' 'u' 'u' 'u']]
maaryasiddiqui@Maaryas-MacBook-Air a3 %

```

SARSA Analysis

Over 5000 episodes, the total time steps the agent took was 64,723 before converging. SARSA achieved a normalized step count of ~1.45, meaning the agent required about 1.45 moves for every unit of Manhattan distance to the goal. Compared to Monte Carlo's normalized step count of 17.5, SARSA is approximately 91.7% more efficient, requiring only 8.3% of the steps Monte Carlo needed per unit of Manhattan distance.

This efficiency increase can be attributed to SARSA's bootstrapping ability, as it

updates Q-values at each step, rather than at the end of the episode, allowing it to correct the policy much faster.

Furthermore, the computation time for all 5000 episodes was ~0.97 seconds, which is far quicker than Monte Carlo. The normalized time of 0.000022s reflects the minimal per-step computational expense of this algorithm.

```

a3 - zsh - 80x24
maaryasiddiqui@Maaryas-MacBook-Air a3 % python3 q.py
Enter p1: 1.0
Enter p2: 0.0

===== Q-LEARNING RESULTS =====

Number of episodes:          5000
Total Time Steps:            63190
Avg Steps / Manhattan Dist:  1.4093
Computation Time:            1.0876s
Avg Time / Manhattan Dist:   0.000024s

Optimal Policy:
[['d' 'd' 'd' 'd' 'u' 'd' 'r' 'r' 'r' 'u']
 ['r' 'd' 'r' 'd' 'u' 'r' 'r' 'r' 'r' 'u']
 ['r' 'r' 'r' 'r' 'r' 'r' 'r' 'r' 'u' 'l']
 ['r' 'u' 'u' 'u' 'u' 'r' 'r' 'u' 'u' 'l']
 ['u' 'u' 'u' 'u' 'u' 'u' 'u' 'u' 'u']
 ['d' 'r' 'u' 'l' 'u' 'r' 'r' 'u' 'l' 'l']
 ['r' 'r' 'u' 'u' 'u' 'r' 'u' 'u' 'l' 'l']
 ['r' 'u' 'u' 'r' 'r' 'r' 'u' 'u' 'u' 'u']
 ['u' 'u' 'u' 'u' 'u' 'u' 'r' 'u' 'u' 'l']
 ['r' 'u' 'u' 'u' 'u' 'u' 'u' 'u' 'l']]
maaryasiddiqui@Maaryas-MacBook-Air a3 %

```

Q-Learning Analysis

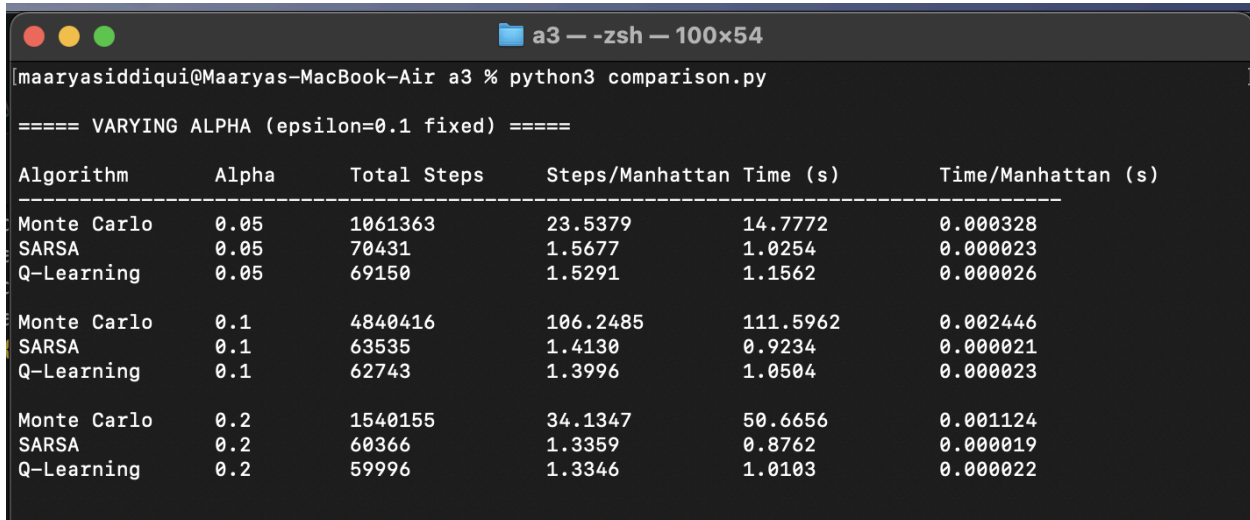
Over 5000 episodes, the total time steps the agent took was 63,190. The agent learned in roughly 1000 fewer steps than SARSA, and significantly faster than Monte-Carlo. Q-Learning achieved a normalized step count of ~1.41, meaning the agent required about 1.41 moves for every unit of Manhattan distance to the goal. Hence, Q-Learning appears to be 2.8% more efficient than SARSA. This can be attributed to Q-learning's updates using greedy max $Q(s', a')$. The computational time was 1.08s, which is slightly longer than SARSA.

Overall, by comparing these three algorithms with $p_1=1.0$, $p_2=0.0$, $\alpha=0.1$, $\gamma=0.9$, $\epsilon=0.1$, Q-Learning and SARSA were both efficient at learning the optimal policy and took the fewest amount of moves for every unit of Manhattan distance to reach the goal.

Q3 - Algorithm Comparison with Varying Alpha, Epsilon = 0.1

The following command was used to view the performance statistics for all three algorithms collectively. Alpha values are tested [0.05, 0.1, 0.2] while $\epsilon=0.1$. 3 runs per algorithm, 5000 episodes per run.

```
python3 comparison.py
```



Algorithm	Alpha	Total Steps	Steps/Manhattan	Time (s)	Time/Manhattan (s)
Monte Carlo	0.05	1061363	23.5379	14.7772	0.000328
SARSA	0.05	70431	1.5677	1.0254	0.000023
Q-Learning	0.05	69150	1.5291	1.1562	0.000026
Monte Carlo	0.1	4840416	106.2485	111.5962	0.002446
SARSA	0.1	63535	1.4130	0.9234	0.000021
Q-Learning	0.1	62743	1.3996	1.0504	0.000023
Monte Carlo	0.2	1540155	34.1347	50.6656	0.001124
SARSA	0.2	60366	1.3359	0.8762	0.000019
Q-Learning	0.2	59996	1.3346	1.0103	0.000022

At each value of alpha, which is the step-size parameter, Monte Carlo significantly underperforms. MC's normalized steps are 15-75 times higher than SARSA and Q-Learning, with the worst being when $\alpha=0.1$ (normalized steps=106.25) meaning the agent took over 100 moves per unit of straight-line distance. This is due to the delayed learning in the Monte Carlo algorithm. It is surprising that $\alpha=0.1$ caused such a significant delay in convergence. However, this shows how a poorly suited step-size parameter can compound errors across the entire episode.

In contrast, both SARSA and Q-Learning show a consistent trend - as alpha increases, the normalized step count, and computational time both decrease. This indicates that with larger values of alpha, and hence larger Q-value updates per step, the algorithm learns faster. The best step-size parameter for both SARSA and Q-Learning is $\alpha=0.2$.

As mentioned previously, Q-Learning marginally outperforms SARSA due to its greedy max $Q(s', a')$ addition in the update step. However, the marginal difference may also be due to random noise across runs.

Q3 - Algorithm Comparison with Varying Epsilon, Alpha = 0.1

The following command was used to view the performance statistics for all three algorithms collectively. Epsilon values are tested [0.05, 0.1, 0.2] while alpha=0.1. 3 runs per algorithm, 5000 episodes per run.

```
python3 comparison.py
```

```
===== VARYING EPSILON (alpha=0.1 fixed) =====
```

Algorithm	Epsilon	Total Steps	Steps/Manhattan	Time (s)	Time/Manhattan (s)
Monte Carlo	0.05	21282328	470.4714	508.3009	0.011233
SARSA	0.05	60215	1.3361	0.8788	0.000020
Q-Learning	0.05	59462	1.3172	1.0966	0.000024
Monte Carlo	0.1	1953559	43.4048	27.0019	0.000600
SARSA	0.1	63356	1.4056	0.9213	0.000020
Q-Learning	0.1	63297	1.4073	1.0610	0.000024
Monte Carlo	0.2	308550	6.8561	4.4569	0.000099
SARSA	0.2	71852	1.5962	1.0951	0.000024
Q-Learning	0.2	70922	1.5769	1.2328	0.000027

A similar trend appears with changes in epsilon values. Monte Carlo consistently underperforms as compared to SARSA and Q-Learning. However, it is worth noting that for Monte Carlo, the number of normalized steps decreases as epsilon increases. Specifically, there is a 10x improvement from epsilon=0.05 to epsilon=0.2. This suggests that more exploration improves the overall performance of the Monte Carlo algorithm. Increased exploration is crucial for Monte Carlo as it doesn't have the ability to correct bad estimates mid-episode. Moreover, Monte Carlo's computational time significantly decreases as more exploration is performed.

However, a difference is noted for SARSA and Q-Learning - as epsilon increases, the normalized step count increases. Though these are not drastic increases, lower epsilon values may be better since both SARSA and Q-Learning can learn efficiently at each step due to bootstrapping. For these algorithms, extra exploration adds unnecessary random moves which increase the step count, without contributing to the efficiency of the algorithms.

Based on this, Monte Carlo and the TD learning algorithms respond inversely to epsilon. Monte Carlo benefits from higher values of epsilon, and hence more exploration. SARSA and Q-Learning benefit from lower values of epsilon, and hence more greedy moves.

Below are the best values of epsilon for each algorithm:

Monte Carlo → epsilon = 0.2 (6.86 normalized steps)

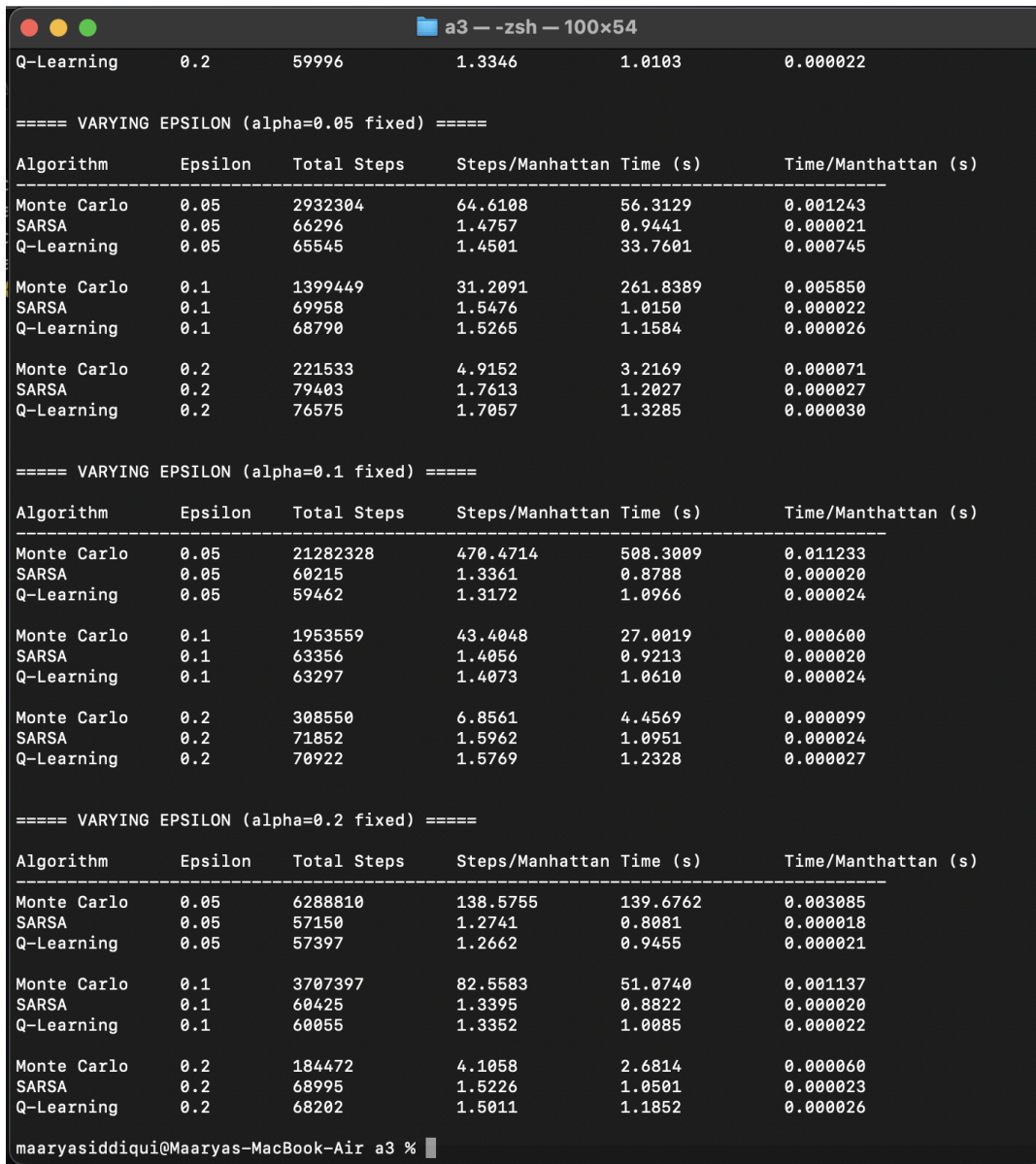
SARSA → epsilon = 0.05 (1.34 normalized steps)

Q-Learning → epsilon = 0.05 (1.32 normalized steps)

Q3 - Algorithm Comparison with Varying Epsilon & Alpha

The following command was used to view the performance statistics for all three algorithms collectively to view the trend for each algorithm as epsilon approaches zero. 3 runs per algorithm, 5000 episodes per run.

```
python3 comparison.py
```



```
a3 --zsh -- 100x54
```

Q-Learning	0.2	59996	1.3346	1.0103	0.000022
------------	-----	-------	--------	--------	----------

===== VARYING EPSILON (alpha=0.05 fixed) =====

Algorithm	Epsilon	Total Steps	Steps/Manhattan	Time (s)	Time/Manhattan (s)
Monte Carlo	0.05	2932304	64.6108	56.3129	0.001243
SARSA	0.05	66296	1.4757	0.9441	0.000021
Q-Learning	0.05	65545	1.4501	33.7601	0.000745
Monte Carlo	0.1	1399449	31.2091	261.8389	0.005850
SARSA	0.1	69958	1.5476	1.0150	0.000022
Q-Learning	0.1	68790	1.5265	1.1584	0.000026
Monte Carlo	0.2	221533	4.9152	3.2169	0.000071
SARSA	0.2	79403	1.7613	1.2027	0.000027
Q-Learning	0.2	76575	1.7057	1.3285	0.000030

===== VARYING EPSILON (alpha=0.1 fixed) =====

Algorithm	Epsilon	Total Steps	Steps/Manhattan	Time (s)	Time/Manhattan (s)
Monte Carlo	0.05	21282328	470.4714	508.3009	0.011233
SARSA	0.05	60215	1.3361	0.8788	0.000020
Q-Learning	0.05	59462	1.3172	1.0966	0.000024
Monte Carlo	0.1	1953559	43.4048	27.0019	0.000600
SARSA	0.1	63356	1.4056	0.9213	0.000020
Q-Learning	0.1	63297	1.4073	1.0610	0.000024
Monte Carlo	0.2	308550	6.8561	4.4569	0.000099
SARSA	0.2	71852	1.5962	1.0951	0.000024
Q-Learning	0.2	70922	1.5769	1.2328	0.000027

===== VARYING EPSILON (alpha=0.2 fixed) =====

Algorithm	Epsilon	Total Steps	Steps/Manhattan	Time (s)	Time/Manhattan (s)
Monte Carlo	0.05	6288810	138.5755	139.6762	0.003085
SARSA	0.05	57150	1.2741	0.8081	0.000018
Q-Learning	0.05	57397	1.2662	0.9455	0.000021
Monte Carlo	0.1	3707397	82.5583	51.0740	0.001137
SARSA	0.1	60425	1.3395	0.8822	0.000020
Q-Learning	0.1	60055	1.3352	1.0085	0.000022
Monte Carlo	0.2	184472	4.1058	2.6814	0.000060
SARSA	0.2	68995	1.5226	1.0501	0.000023
Q-Learning	0.2	68202	1.5011	1.1852	0.000026

maaryasiddiqui@Maaryas-MacBook-Air a3 %

Across all three epsilon-varying sections, a consistent pattern emerges as epsilon decreases towards 0.05 (approaches 0) - Monte Carlo's performance degrades. This confirms that Monte Carlo's performance is dependent on exploration to compensate for its inability to learn during the course of an episode.

However, in contrast the TD algorithms, SARSA and Q-Learning, both improve as epsilon approaches 0.05, and hence as epsilon decreases. This is because both algorithms perform bootstrapping and already perform learning updates at each step, hence forced exploration is not helpful. The best results for both SARSA and Q-Learning are noted at $\alpha=0.2$, $\epsilon=0.05$, where the normalized steps are 1.27 and 1.26 respectively.

What's interesting to note is that for SARSA and Q-Learning, higher values of α compensate for higher exploration (larger epsilon values). The penalty of increasing epsilon, in terms of normalized steps, shrinks as α increases. This suggests that the faster Q-value updates due to a higher α value means the agent can recover more quickly from random exploratory moves.

Q3 - Conclusion

In conclusion, both TD learning methods outperform Monte Carlo, with Q-Learning being marginally better than SARSA. Their ability to bootstrap and learn mid-episode, instead of at the end of each episode, makes them far more efficient than Monte Carlo. These TD methods perform optimally with larger step-size parameters, and smaller epsilon values to decrease exploration, and increase exploitation via bootstrapping.